

SYSTEM IMPLEMENTATION

5.1 Sample code

```
1. Main.py from fastapi import FastAPI from fastapi.middleware.cors import
CORSMiddleware from app.db.session import engine, Base from app.models.user
import User from app.models.user_progress import UserProgress from
app.models.result import Result from app.models.resume import Resume from
app.models.skill import Skill from app.models.question import Question from
app.models.report import EvaluationReport from app.models.test import Test from
app.models.interview_session import InterviewStatus, InterviewRound from
app.models.cheating_log import CheatingLog from app.api.v1.endpoints import
auth, resume, practice, interview, results, proctor from app.api.v1.endpoints import
users

# Create all database tables (if they don't exist yet)
Base.metadata.create_all(bind=engine)

# Initialize the FastAPI App (CREATE ONLY ONCE!)
app = FastAPI(    title="AI Virtual Mock Interview
System",

    description="Backend API for Resume Analysis, Safe-Zone Practice, and Proctored Real
Interviews",    version="3.0.0"

# CORS CONFIGURATION (CRUCIAL FOR REACT & WEBSOCKETS)
origins = [
    "http://localhost:5173",
    "http://localhost:5174",
    "http://localhost:5175",
    "http://127.0.0.1:5173",
    "http://127.0.0.1:5174",
    "http://127.0.0.1:5175",
    "http://localhost:3000",
```

```

    "http://127.0.0.1:3000",
]
# This MUST come immediately after app = FastAPI()
app.add_middleware(    CORSMiddleware,
allow_origins=origins,    allow_credentials=True,
allow_methods=["*"], # Allows POST, GET, OPTIONS, etc.
    allow_headers=["*"], # Allows Authorization headers
)
# REGISTER ALL ROUTERS
app.include_router(auth.router, prefix="/api/v1/auth", tags=["Authentication"])
app.include_router(users.router, prefix="/api/v1/users", tags=["Users"])
app.include_router(resume.router, prefix="/api/v1/resume", tags=["Resume Analysis"])
app.include_router(practice.router, prefix="/api/v1/practice", tags=["Practice Arena"])
app.include_router(interview.router, prefix="/api/v1/interview", tags=["Real Interviews"])
app.include_router(proctor.router, prefix="/api/v1/proctor", tags=["Anti-Cheating API"])
# app.include_router(results.router, prefix="/api/v1/results", tags=["Final Results"])
@app.exception_handler(Exception)    async    def
global_exception_handler(request: Request, exc: Exception):
    print(" GLOBAL ERROR:")
    traceback.print_exc()
    return {"detail": str(exc)}
import React, {
useState, useEffect } from 'react'; import {
useNavigate } from 'react-router-dom'; import {
motion } from 'framer-motion';
import { LineChart, Line, BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, Legend,
ResponsiveContainer } from 'recharts'; import toast from 'react-hot-
toast'; // Added for the locked notification import { useAuthStore
} from '../store'; import { userService, resumeService } from '../api';
import { StatCard, CircularProgress } from '../components/features';
import { Button, Card, Skeleton } from '../components/ui'; void

```

```

motion; const Dashboard = () => {  const navigate =
useNavigate();  const user = useAuthStore((state) => state.user);
const logout = useAuthStore((state) => state.logout);

  // State for fetched data  const [dashboardData,
setDashboardData] = useState(null);  const [resumeAnalysis,
setResumeAnalysis] = useState(null);  const [loading,
setLoading] = useState(true);  const [error, setError] =
useState(null)  const performanceData = [
    { name: 'Week 1', score: 65, target: 80 },
    { name: 'Week 2', score: 72, target: 80 },
    { name: 'Week 3', score: 78, target: 80 },
    { name: 'Week 4', score: 85, target: 80 },
    { name: 'Week 5', score: 88, target: 80 },
  ];

  useEffect(()    =>    {          const
fetchDashboardData = async () => {
    try {
      setLoading(true);
      setError(null);

      if (user?.id) {
        try {
          const statsResponse    =    await    userService.getDashboardStats(user.id);
          setDashboardData(statsResponse);
        } catch {          console.warn('Dashboard stats endpoint not available,
using defaults');
        }
      }
    }
  }
}

```

```

        const resumeResponse = await resumeService.analyze(user.id);
setResumeAnalysis(resumeResponse);

        } catch { console.warn('Resume analysis endpoint
not available');

        }

        }

        } catch (err) { console.error('Error fetching
dashboard data:', err); setError(err.message);

        } finally {

setLoading(false);

        }

    };

    fetchDashboardData();
}, [user?.id]); const dashboardStats = dashboardData?.data ||
dashboardData || {}; const storedResumeData = resumeAnalysis?.data ||
resumeAnalysis || {};

const normalizeRoundScore = (score, round) => { const numericScore = Number(score ||
0); if (!Number.isFinite(numericScore) || numericScore <= 0) return 0; if (round === 'mcq'
&& numericScore <= 30) return Math.round((numericScore / 30) * 100); if (round ===
'theory' && numericScore <= 5) return Math.round(numericScore * 20); if (round !==
'coding' && numericScore <= 10) return Math.round(numericScore * 10); return
Math.min(Math.round(numericScore), 100);

}

const completedAverage = (...scores) => { const
completedScores = scores.filter((score) => score > 0); if
(!completedScores.length) return 0;

return Math.round(completedScores.reduce((sum, score) => sum + score, 0) /
completedScores.length);

};

```

```

const normalizedScores = {   mcq:
normalizeRoundScore(dashboardStats?.mcq_score, 'mcq'),   theory:
normalizeRoundScore(dashboardStats?.theory_score, 'theory'),

      communication:   normalizeRoundScore(dashboardStats?.communication_score,
'communication'),                                     coding:
normalizeRoundScore(dashboardStats?.coding_score, 'coding'),

};

const atsScore = storedResumeData?.ats_score || dashboardStats?.ats_score || 0; const
practiceTests = dashboardStats?.practice_tests_completed || 0;

const completedPracticeAverage = completedAverage(
normalizedScores.mcq, normalizedScores.theory,
normalizedScores.communication,
normalizedScores.coding
);

      const      interviewScore      =      completedPracticeAverage      ||
normalizeRoundScore(dashboardStats?.interview_score,      'interview');

const currentStreak = dashboardStats?.current_streak || 0;

// DEMO OVERRIDE LOGIC

const isDemoAccount = user?.email === 'demo@gmail.com' || user?.login_id ===
'demo@gmail.com'; const hasPassedPractice = interviewScore >= 80;

const canAccessRealInterview = hasPassedPractice || isDemoAccount;

const testData = [
  { name: 'MCQ', accuracy: normalizedScores.mcq },
  { name: 'Coding', accuracy: normalizedScores.coding },
  { name: 'Theory', accuracy: normalizedScores.theory },
  { name: 'Comm', accuracy: normalizedScores.communication },
];

```

```

const recentTests = dashboardStats?.recent_tests?.map((test) => {
const round = test.name?.toLowerCase().includes('mcq') ?
'mcq' : test.name?.toLowerCase().includes('theory') ? 'theory' :
test.name?.toLowerCase().includes('communication') ? 'communication' :
test.name?.toLowerCase().includes('coding') ? 'coding' :
'score';

return {
...test,
const targetPath = roundPath[currentRound] || '/practice';

navigate(targetPath, {
state: {
extractedSkills:
resumeAnalysis?.data?.extracted_skills ||
resumeAnalysis?.extracted_skills ||
[],
skillToTest: resumeAnalysis?.data?.extracted_skills?.[0] || 'General',
domain: 'Technology'
},
});
});

// : View Report Button Logic const
handleViewReport = () => {
// Navigate straight to the Overall Report step navigate('/interview-
results', { state: { forceOverallReport: true } });
});
return (
<div className="min-h-screen bg-dark-900 py-8 px-4 md:px-8">

```

```

<div className="max-w-7xl mx-auto">

  {error && (
    <motion.div
      initial={{ opacity: 0, y: -20 }}
      animate={{ opacity: 1, y: 0 }}
      className="mb-6 p-4 bg-red-500/10 border
border-red-500/30 rounded-lg"

      <p className="text-red-400 text-sm"> {error}</p>

    </motion.div>

  )}

  <motion.div

    initial={{ opacity: 0, y: -20 }}
    animate={{
      opacity: 1, y: 0 }}
    className="flex justify-between
items-center mb-12"

    >

    <div>

      <h1 className="text-4xl md:text-5xl font-bold text-white mb-2">

        Welcome back, {user?.name || 'Champion'}!

      </h1>

      <p className="text-gray-400 text-lg">Ready to ace your next interview?</p>

    </div>

    <Button variant="ghost" onClick={handleLogout}>

      Logout

    </Button>

  </motion.div>

  <motion.div
    initial={{ opacity: 0, y: 20 }}
    animate={{ opacity: 1, y: 0 }}
    transition={{ staggerChildren: 0.1,
      delayChildren: 0.1 }}
    className="grid grid-cols-1 md:grid-cols-2
lg:grid-cols-4 gap-6 mb-12"

    >

    {loading ? (
      Array(4).fill(0).map((_, i) => (
        <Skeleton key={i} className="h-32" />

```

```

    ))
  ): (
    <<StatCard title="ATS Score" value={atsScore} unit="/100"
trend={dashboardStats?.ats_trend || 0} color="blue" icon=" " />

    <StatCard title="Practice Tests" value={practiceTests} unit="completed"
trend={dashboardStats?.practice_trend || 0} color="purple" icon=" " />

    <StatCard title="Interview Score" value={interviewScore} unit="average"
trend={dashboardStats?.interview_trend || 0} color="green" icon=" " />

    <StatCard title="Current Streak" value={currentStreak} unit="days"
trend={dashboardStats?.streak_trend || 0} color="orange" icon=" " />

  </>
)}
</motion.div>

<motion.div initial={{ opacity: 0, y: 20 }}
animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.3 }}
className="grid grid-cols-1 lg:grid-cols-3 gap-6 mb-12"
>
  <Card className="lg:col-span-2">
    <h3 className="text-xl font-semibold text-white mb-6">Weekly Performance</h3>
    <ResponsiveContainer width="100%" height={300}>
      <LineChart data={performanceData}>
        <CartesianGrid stroke="rgba(255,255,255,0.1)" />
        <XAxis stroke="rgba(255,255,255,0.5)" />
        <YAxis stroke="rgba(255,255,255,0.5)" />
        <Tooltip contentStyle={{
background: 'rgba(15, 23, 42, 0.95)',
border: '1px solid rgba(255,255,255,0.1)',
borderRadius: '8px',
}}
/>

```



```

    <Legend />

    <Line type="monotone" dataKey="score" stroke="#3b82f6" strokeWidth={2}
dot={{ fill: '#3b82f6' }} />

    <Line type="monotone" dataKey="target" stroke="#10b981" strokeWidth={2}
strokeDasharray="5 5" />

  </LineChart>

</ResponsiveContainer>

</Card>

<Card className="flex flex-col items-center justify-center">

  <h3 className="text-xl font-semibold text-white mb-6">Your ATS Score</h3>

  {loading ? (
    <Skeleton className="w-32 h-32 rounded-full" />
  ) : (
    <>
    <CircularProgress value={atsScore} size="lg" />

    <p className="text-gray-300 mt-4 text-center">
      {atsScore >= 80 ? (
        <>Your resume ranks in the <span className="text-green-400 font-
semibold">top 15%</span></>
      ) : atsScore >= 60 ? (
        <>Your resume is <span className="text-yellow-400 font-semibold">pretty
good</span></>
      ) : (
        <>Upload a resume to get an <span className="text-blue-400
fontsemibold">ATS Score</span></>
      )}
    </p>
    </>
  )}
</Card>

```

```
</motion.div>
```

```
    <motion.div      initial={{ opacity: 0, y: 20 }}
    animate={{ opacity: 1, y: 0 }}      transition={{ delay: 0.4 }}
    className="grid grid-cols-1 lg:grid-cols-3 gap-6 mb-12"
    >
      <Card className="lg:col-span-2">
        <h3 className="text-xl font-semibold text-white mb-6">Category Accuracy</h3>
        <ResponsiveContainer width="100%" height={250}>
          <BarChart data={testData}>
            <CartesianGrid stroke="rgba(255,255,255,0.1)" />
            <XAxis      stroke="rgba(255,255,255,0.5)"      />
            <YAxis stroke="rgba(255,255,255,0.5)" />
            <Tooltip      contentStyle={{
background: 'rgba(15, 23, 42, 0.95)',
border: '1px solid rgba(255,255,255,0.1)',
borderRadius: '8px',
      }}
            />
            <Bar dataKey="accuracy" fill="#8b5cf6" radius={[8, 8, 0, 0]} />
          </BarChart>
        </ResponsiveContainer>
      </Card>

      <Card className="flex flex-col gap-4">
        <h3 className="text-xl font-semibold text-white">Quick Actions</h3>
        <Button variant="primary" onClick={handleResume} className="w-full">
          Upload Resume
        </Button>
```

```

<Button variant="primary" onClick={handlePractice} className="w-full">
  Practice Test
</Button>

{/* DYNAMIC BUTTON: Locked for students < 80%, open for HOD */}
<Button          variant={canAccessRealInterview ?
"success" : "secondary"}          onClick={() => {          if
(canAccessRealInterview) {          navigate('/interview');
          } else {
            toast.error("You must score 80% average in practice to unlock the Real Interview.",
{
            style: { background: '#111827', color: '#ef4444', border: '1px solid #ef4444'
}
            });
          }
        }}
        className={`w-full text-base font-semibold ${!canAccessRealInterview ? 'opacity60
cursor-not-allowed' : ''}`}
    >
      {canAccessRealInterview ? "Real Interview" : "Score 80% to Unlock"}
</Button>

{/* FIXED: View Report Button now links to the Overall Practice Report */}
<Button variant="secondary" onClick={handleViewReport} className="w-full">
  View Reports
</Button>
</Card>
</motion.div>

<motion.div initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{
delay: 0.5 }}>

```

```

<Card>

  <h3 className="text-xl font-semibold text-white mb-6">Recent Tests</h3>

  <div className="space-y-4">

    {loading ? (
      Array(3).fill(0).map((_, i) => <Skeleton key={i} className="h-16" />)
    ) : recentTests.length > 0 ? (
      recentTests.map((test, index) => (
        <motion.div
          <div>
            <p className="text-white font-semibold">{test.name}</p>
            <p className="text-gray-400 text-sm">{test.date}</p>
          </div>
          <div className="text-right">
            <p className="text-white font-bold text-lg">{test.score}%</p>
            <p className="text-gray-400 text-sm">Score</p>
          </div>
        </motion.div>
      ))
    ) : null}

  </div>
</div>

);
export default
Dashboard;

```

Steps for Execution

Step 0: Install required software and technologies including React.js, Python, Fast API, PostgreSQL, and necessary AI libraries.

Step 01: Create the frontend application using React.js for user interface, dashboard, resume upload, practice rounds, and interview sections.

Step 02: Develop the backend system using Fast API (Python) to handle authentication, resume processing, API routing, and evaluation logic.

Step 03: Configure PostgreSQL database to store user details, resumes, ATS scores, extracted skills, test results, and final reports.

Step 04: Implement User Registration and Login module with secure authentication for accessing the platform.

Step 05: Develop Resume Upload and Resume Analysis module using NLP for ATS score calculation, skill extraction, and domain assignment.

Step 06: Build the dashboard to display ATS score, extracted skills, missing skills, feedback, and domain suggestions.

Step 07: Develop the Practice Sandbox module with four rounds: MCQ, Theory, Communication, and Coding.

Step 08: Integrate AI models for question generation, speech-to-text processing, and user response evaluation.

Step 09: Implement eligibility verification to check whether the user meets the minimum performance threshold (80%) for real interview access.

Step 10: Develop the Real Interview module with advanced interview rounds, strict timers, and controlled testing conditions.

Step 11: Implement anti-cheat and monitoring features for secure interview execution.

Step 12: Develop the Evaluation & Recommendation module to calculate final scores, generate reports, and recommend qualified candidates to startup companies.